

A Lab Report
On

“Implementation of Object Oriented Concepts and Design Patterns in Java”

For The Course

“Object Oriented Pattern and Design Lab”

Course Code : ICT 2208

Submitted By

Protham Sarker (IT23027)

Submitted To

Dr. Ziaur Rahman

Professor

Department of ICT, MBSTU



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902, DHAKA, BANGLADESH



L1- Encapsulation and Polymorphism

1.1 Encapsulation:

Encapsulation is the process of wrapping data (variables) and methods into a single class and restricting direct access to the data.

Java achieves encapsulation by:

1. Declaring variables private
2. Using public getters
3. Using public setters and
4. Applying validation when needed.

```
class Student {
```

```
    private double cgpa;
```

```
    public double getCgpa() {
```

```
        return cgpa;
```

```
    }
```

```
    public void setCgpa(double cgpa) {
```

```
        if (cgpa >= 0 && cgpa <= 4.0)
```

```
            this.cgpa = cgpa;
```

```
    } else
```

```
        System.out.println
```

```
            ("Invalid CGPA");
```

```
    }
```

```
}
```

1.2 Polymorphism:

Polymorphism means 'many forms'. It allows the same method name or reference to behave differently in different situations.

Compile time Polymorphism	Run-time Polymorphism
Achieved by method overloading	Achieved by method overriding
Usually within the same class	Parent-child class
Parameters are different	Method signature is the same
Decided by compiler	Decided by JVM at runtime
Static binding	Dynamic / Late binding

Overloading example:

```
class Calculator {
```

```
    int add (int a, int b) {return  
                                a+b; }
```

```
}
```

Overriding example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal  
            makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
Animal a = new Dog();  
a.sound(); // Dog barks
```

1.3 Practical - BankAccount

```
class BankAccount {  
    private double balance;  
    public double getBalance() {  
        return balance;  
    }  
    public void deposit (double amount) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println ("Deposited: " + amount);  
        } else {  
            System.out.println ("Invalid amount");  
        }  
    }  
    public void deposit (double amount,  
        String remarks) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println ("Deposited: " + amount);  
            System.out.println ("Remarks: " + remarks);  
        } else {  
            System.out.println ("Invalid amount");  
        }  
    }  
}
```

```
public class Main {  
    public static void main (String[] args) {  
        BankAccount acc = new BankAccount (1000);  
        acc.deposit (500);  
        acc.deposit (200, "salary");  
        System.out.println ("Final Balance: "  
            + acc.getBalance () );  
    }  
}
```

This demonstrates encapsulation because balance is private and polymorphism because deposit() is overloaded.

L2 - Method Overloading vs Overriding

2.1

Feature	Method Overloading	Method Overriding
Meaning	Same method name with different parameters	Child class redefines parent method
Classes	Usually same class	parent + child class
Inheritance	Not required	Required
Parameters	Must be different	Must be same
Return Type	Cannot distinguish methods by return type alone	Same or covariant
Binding	Compile time	Runtime

2.2 Early Binding vs Late Binding

Early Binding	Late Binding
Compile time	Runtime
Mainly overloading	Mainly overriding
Method call determined by compiler	Method call determined during execution
Static binding	Dynamic Binding

2.3 Practical - Shape Hierarchy

```

class Shape {
    double area() {return 0;}
    void describe (String name) {
        System.out.println("Shape: "+name);
    }
    void describe (String name, int sides) {
        System.out.println("Shape: "+ name +
            sides: "+ sides);
    }
}

class Circle extends Shape {
    double radius;
    Circle (double radius) {this.radius=radius;}

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, width;
    Rectangle (double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    double area() {
        return length * width;
    }
}

```

```

public class Main {
    public static void main (String [] args) {
        Shape s1 = new Circle (5);
        Shape s2 = new Rectangle (4, 6);
        s1.describe ("Circle"); //overloading
        System.out.println ("Circle Area: " + s1.area()); //overriding
        s2.describe ("Rectangle", 4); //overloading
        System.out.println ("Rectangle Area: "
            + s2.area()); //overriding
    }
}

```

Feature	Abstract Class	Interface
Instantiation	Can have	Cannot
Variables	Can have	No ordinary instance fields; all are public static final
Methods	Abstract + concrete	Abstract + default state and more modern interface features
Multiple inheritance	Class extends only one class	Class can implement multiple interfaces
Usage	Common identity, shared code	Capability, contract

L3- Abstract Class vs Interface

3.1 Definitions & Differences

Abstract Class: A class declared with abstract that cannot be instantiated directly. It may contain abstract and concrete methods, instance variables and constructors.

Interface: A contract that a class agrees to implement. Its fields are implicitly public static final constants; modern Java also supports default and static methods

Feature	Abstract class	Interface
Declaration	abstract class	interface
Constructor	Can have	Cannot
Instance variables	Can have	No ordinary instance fields; fields are public static final
Methods	Abstract + concrete	Abstract + default/ static and more modern interface features
Multiple inheritance	Class extends only one class	Class can implement multiple interfaces
purpose	Common identity/ shared code	Capability/ contract

3.2 When to choose which?

Abstract class: Use when related classes share common properties and implementation.
e.g., vehicle → car, bike, truck.

Interface: Use when different classes need a common capability, e.g., Car and House can both be Insurable.

3.3 Practical - vehicle / Insurable / car

```
abstract class Vehicle {
    void startEngine() {
        System.out.println("Engine started");
    }
    abstract String fuelType();
}

interface Insurable {
    double calculatePremium();
}

class Car extends Vehicle implements Insurable {
    @Override
    String fuelType() {
        return "Petrol";
    }
}
```

@Override

```
public double calculatePremium() {  
    return 500.0;  
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Car car = new Car();
```

```
        car.startEngine();
```

```
        System.out.println("Fuel Type: " +  
                             car.fuelType());
```

```
        System.out.println("Premium: " +  
                             car.calculatePremium());
```

```
    }
```

```
}
```

L4 - Java Collection Framework

4.1 ArrayList vs Vector vs LinkedList

Feature	ArrayList	Vector	LinkedList
Structure	Dynamic array	Dynamic array	Doubly linked list
Thread-safe	No	Yes (synchronized)	No
Random access	Fast $O(1)$	Fast $O(1)$	Slow $O(n)$
Middle insert/delete	Usually $O(n)$	Usually $O(n)$	Efficient after reaching position
Use	General-purpose list	Legacy synchronized list	Frequent insertion / deletion use cases

4.2 Set and Its Implementations

Set: A collection that does not allow duplicate elements

HashSet	Linked HashSet	TreeSet
No duplicates	No duplicates	No duplicates
No guaranteed order	Insertion order	Sorted order
Generally fastest average operations	Slight extra overhead	Generally slower tree based
Hash table	Hash table + linked structure	Balanced search tree

4.3 Practical - ArrayList vs TreeSet

```
import java.util. ArrayList ;
import java.util. TreeSet;
public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Rafi");
        list.add("Mim");
        list.add("Zain");
        list.add("Alina");
        list.add("Karim");

        System.out.println("ArrayList: ");
        for (String name : list)
            System.out.println(name);

        TreeSet<String> set = new TreeSet<>(list);

        System.out.println("\nTreeSet: ");
        for (String name: set)
            System.out.println(name);
    }
}
```

L5 - Multithreading & Exception Handling

5.1 Ways to Create Threads →

1. Extend Thread and override run().
2. Implement Runnable and pass it to Thread
3. Use Executor Service for thread-pool based task management.

Extending Thread	Implementing Runnable	Executor Service
Simple	Flexible	Best for managing many tasks
Class must extend Thread	Class can extend another class	Uses Thread pool
Less reusable	More reusable	Efficient resource management

5.2 Practical - Two Threads

```

class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++)
            System.out.println("Thread 1: " + i);
    }
}
  
```

```

class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++)
            System.out.println("Thread 2 : " + i);
    }
}

```

```

}
public class Main {
    public static void main (String[] args) {
        MyThread t1 = new MyThread();
        Thread t2 = new Thread(new MyRunnable());
        t1.start();
        t2.start();
    }
}

```

5.3 Custom Exception

A custom exception is a user-defined exception created by extending Exception or RuntimeException.

```

import java.util.Scanner;

class InvalidRadiusException extends Exception {
    public InvalidRadiusException(String message) {
        super(message);
    }
}

```

```

class Circle {
    private double radius;
    public Circle(double radius) throws
        InvalidRadiusException {
        if (radius < 0)
            throw new InvalidRadiusException("
                Radius cannot be negative");
        this.radius = radius;
    }
    public double area() {
        return Math.PI * radius * radius;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter radius: ");
        double r = sc.nextDouble();
        try {
            Circle c = new Circle(r);
            System.out.println("Area: " + c.area());
        } catch (InvalidRadiusException e) {
            System.out.println("Error: " + e.
                getMessage());
        }
        sc.close();
    }
}

```

L6 - JDBC + MVC

6.1 Steps of JDBC connection -

Driver → Connection → PreparedStatement → Execute → ResultSet → Close

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

```
Connection con =
```

```
    DriverManager.getConnection(url, user, password);
```

```
PreparedStatement ps =
```

```
    con.prepareStatement(sql);
```

```
ps.executeUpdate(); // Insert / Update / Delete
```

```
ResultSet rs =
```

```
    ps.executeQuery(); // select
```

6.2 MVC Pattern

MVC separates an application into Model, View and Controller. A DAO is a separate data-access component and should not simply be labeled as the MVC Controller.

Component	Main Job	Example
Model	Represents data/business object	Student
View	User interface/presentation	JSP/JavaFX/HTML
Controller	Handles request and controls flow	StudentServlet
DAO	Performs database operations	StudentDAO

5.3 Practical - MVC Student Insert

// Model

```
class Student {  
    private int id;  
    private String name;  
    private double cgpa;  
    public Student(int id, String name, double cgpa) {  
        this.id = id;  
        this.name = name;  
        this.cgpa = cgpa;  
    }  
    public int getID() { return id; }  
    public String getName() { return name; }  
    public double getGpa() { return cgpa; }  
}
```

// DAO

```
import java.sql.*;  
class StudentDAO {  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/student_db";  
    private static final String USER = "root";  
    private static final String PASS = "password";  
    public void insert(Student s) {  
        String sql =  
            "INSERT INTO students (id, name, cgpa)  
            VALUES (?, ?, ?)";  
    }  
}
```

```
try (Connection con =  
    DriverManager.getConnection(URL, USER,  
        PASS);  
    PreparedStatement ps =  
        con.prepareStatement(sql)) {  
    ps.setInt(1, s.getId());  
    ps.setString(2, s.getName());  
    ps.setDouble(3, s.getCgpa());  
    ps.executeUpdate();  
    System.out.println("Student inserted  
        successfully");  
} catch (SQLException e) {  
    System.out.println("Database error: "  
        + e.getMessage());  
}  
}
```

```
} // view / main
```

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter ID: ");
```

```
        int id = sc.nextInt();
```

```
System.out.print("Enter Name: ");  
String name = sc.next();  
System.out.print("Enter CGPA: ");  
double cgpa = sc.nextDouble();  
Student student = new Student(id,  
                                name, cgpa);  
StudentDAO dao = new StudentDAO();  
dao.insert(student);  
sc.close();  
}
```

```
}
```

L7 - JavaFX House Loan Calculator

7.1 JavaFX Application Structure

Application: Base class for a JavaFX application

Stage: Main window

Scene: Contains the UI content inside a stage.

GridPane: Arranges controls in rows and columns

start(): Main JavaFX lifecycle method where the GUI is created and shown.

Structure: Stage → Scene → Layout → Controls

7.2 & 7.3 Practical - Complete Loan Calculator.

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class LoanCalculator extends Application {
    private TextField loanAmountField =
        new TextField();

    private TextField rateField = new TextField();
    private TextField yearsField = new TextField();
```

```

private Label moneyLabel = new Label ("
                                Monthly Payment: ");
private Label totalLabel = new Label (" Total
                                Payment: ");
private Label interestLabel = new Label ("
                                Total Interest: ");

```

@Override

```

public void start (Stage primaryStage) {
    GridPane grid = new GridPane();
    grid.setPadding(new Insets(15));
    grid.setHgap(10);
    grid.setVgap(10);
    grid.add(new Label("Loan Amount: ", 0, 0);
    grid.add(loanAmountField, 1, 0);
    grid.add(new Label("Annual Rate(%): ", 0, 1);
    grid.add(rateField, 1, 1);
    grid.add("Number of Years: ", 0, 2);
    grid.add(yearsField, 1, 2);
    Button button = new Button("Calculate");
    grid.add(button, 1, 3);
    grid.add(monthlyLabel, 0, 4, 2, 1);
    grid.add(totalLabel, 0, 5, 2, 1);
    grid.add(interestLabel, 0, 6, 2, 1);
}

```

```
button.setOnAction (e -> calculateLoan());
Scene scene = new Scene (grid, 400, 300);
primaryStage.setTitle ("House Loan Calculator");
primaryStage.setScene (scene);
primaryStage.show();
}

private void calculateLoan() {
    double loan = Double.parseDouble (loanAmountField.getText());
    double rate = Double.parseDouble (rateField.getText());
    int years = Integer.parseInt (yearsField.getText());
    double monthlyRate = rate / 100 / 12;
    int months = years * 12;
    double monthlyPayment;
    if (monthlyRate == 0) {
        monthlyPayment = loan / months;
    }
    else {
        monthlyPayment = loan * monthlyRate *
            Math.pow (1 + monthlyRate, months) /
            (Math.pow (1 + monthlyRate, months) - 1);
    }
}
```

```
double totalPayment = monthlyPayment * months;  
double totalInterest = totalPayment - loan;
```

```
monthlyLabel.setText(  
    String.format("Monthly Payment: %.2f",  
        monthlyPayment));
```

```
totalLabel.setText(  
    String.format("Total Payment: %.2f",  
        totalPayment));
```

```
interestLabel.setText(  
    String.format("Total Interest: %.2f",  
        totalInterest));
```

```
    }  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

1.8 - Socket Programming & Java RMI

8.1 Socket Programming vs Java RMI

Socket Programming	Java RMI
Low-level communication	High-level remote object communication
Uses sockets and streams	Invokes methods on remote objects
Programmer manages messages	RMI handles much of the communication
Can work with different languages / protocols	Primarily Java-to-Java
Good for chat / custom protocols	Good for distributed Java applications

8.2 Server Code

```
import java.io.*;
```

```
import java.net.*;
```

```
public class ChatServer {
```

```
    public static void main(String[] args) throws  
        IOException {
```

```
        ServerSocket serverSocket = new ServerSocket(500);
```

```
        System.out.println("Server waiting...");
```

```

socket socket = serverSocket.accept();
system.out.println("Client connected!");
BufferedReader in =
    new BufferedReader(
        new InputStreamReader(socket.
            getInputStream()));

```

```

PrintWriter out =
    new PrintWriter(socket.getOutputStream(),
        true);

```

```

String message = in.readLine();
system.out.println("Client says: " + message);
out.println("Message received: " + message);

```

```

socket.close();
serverSocket.close();

```

```

}

```

```

}

```

8.3 Client Code -

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

```

```

public class ChatClient {
    public static void main(String[] args) throws
        IOException {
        Socket socket = new Socket("localhost", 5000);
    }
}

```

```
PrintWriter out =  
    new PrintWriter(Socket.getOutputStream(),  
                     true);
```

```
BufferedReader in =  
    new BufferedReader(  
        new InputStreamReader(Socket.  
            getInputStream()));
```

```
Scanner sc = new Scanner(System.in);  
System.out.print("Enter message: ");
```

```
String message = sc.nextLine();  
out.println(message);
```

```
String reply = in.readLine();  
System.out.println("Server reply: " +  
                    reply);
```

```
socket.close();  
sc.close();
```

```
}
```

```
}
```

1. Project Setup

```
CREATE DATABASE student_db;
USE student_db;
CREATE TABLE Students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    cgpa DOUBLE
);
```

Typical flow: Browser/JSP Form → Servlet → DAO/JDBC → MySQL Database

9.2 Servlet doPost() -

```
import java.io.IOException;
import java.sql.*;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {

    @Override
    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        int id =
            Integer.parseInt(request.getParameter(
                "id"));
```

29

```

string name =
    request.getParameter("name");
double cgpa =
    Double.parseDouble(request.getParameter("cgpa"));
string url =
    "jdbc:mysql://localhost:3306/student_db";
string user = "root";
string pass = "password";
string sql =
    "INSERT INTO students (id, name, cgpa)
    VALUES (?, ?, ?)";
try (Connection con =
    DriverManager.getConnection(url,
    user, pass);

    PreparedStatement ps =
        con.prepareStatement(sql)) {
    ps.setInt(1, id);
    ps.setString(2, name);
    ps.setDouble(3, cgpa);
    ps.executeUpdate();
    response.getWriter().println("Student
    added successfully!");
} catch (SQLException e) {
    response.getWriter().println("Database
    Error: " + e.getMessage());
}
}

```

3 JSP - Display Student Records

```
%@ page import = "java.sql.*" %>
```

```
html>
```

```
head>
```

```
<title> Student Records </title>
```

```
/head>
```

```
body>
```

```
<h2> Student Records </h2>
```

```
<table border = "1" >
```

```
<tr>
```

```
<th> ID </th>
```

```
<th> Name </th>
```

```
<th> CGPA </th>
```

```
</tr>
```

```
<%
```

```
String url =
```

```
"jdbc:mysql://localhost:3306/studentdb;
```

```
try (Connection con =
```

```
DriverManager.getConnection(url,  
user, pass);
```

```
PreparedStatement ps =
```

```
con.prepareStatement(sql);
```

```
ResultSet res =
```

```
ps.executeQuery() {
```

```
while (rs.next()) {
```

```
%>
```

```

</tr>
<td><%=rs.getInt("Id")%></td>
<td><%=rs.getString("Name")%></td>
<td><%=rs.getDouble("cgpa")%></td>

```

```

</tr>

```

```

<%

```

```

}

```

```

} catch (SQLException e) {

```

```

%>

```

```

<tr>

```

```

<td colspan="3">

```

```

Database Error : <%=e.getMessage()%>

```

```

</td>

```

```

</tr>

```

```

<%

```

```

}

```

```

%>

```

```

</table>

```

```

</body>

```

```

</html>

```

10.1 Setting up Spring Boot with REST and JPA

Below is the sequence used to scaffold a Spring Boot service backed by REST endpoints and Spring Data JPA.

Step 1 - Generate the project. Use Spring Initializr (start.spring.io) and choose Maven as the build tool, Java as the language, and add the following starters: *spring-boot-starter-web*, *spring-boot-starter-data-jpa*, and the MySQL driver.

Step 2 - Declare dependencies in pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

Step 3 - Point the app at the database inside application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/campusdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Step 4. Write an entity class annotated with @Entity, a repository interface, and a REST controller that exposes it.

Step 5. Launch the application from the class carrying @SpringBootApplication.

What the embedded Tomcat actually does: Spring Boot ships an embedded Tomcat instance inside the runnable JAR itself, so there is no separate install step and nothing to deploy into an external servlet container. The moment the application starts, this embedded server boots on a configurable port (port 8080 unless overridden), begins accepting HTTP traffic, and hands each request to the DispatcherServlet, which in turn dispatches it to the matching REST controller method. The net effect is a fully self-contained artifact that can be launched with nothing more than `java -jar app.jar`.

10.2 Entity and Repository Layer

Learner.java — the JPA entity representing a student record;

```
package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "learners")
public class Learner {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String fullName;
    private double gpa;

    public Learner() {}

    public Learner(String fullName, double gpa) {
        this.fullName = fullName;
        this.gpa = gpa;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public double getGpa() { return gpa; }
    public void setGpa(double gpa) { this.gpa = gpa; }
}
```

LearnerRepository.java — a Spring Data JPA repository that gives CRUD operations for free:

```
package com.example.demo.repository;

import com.example.demo.entity.Learner;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface LearnerRepository extends JpaRepository<Learner, Long> {
}
```

0.3 REST Controller

LearnerController.java — exposes the /learners endpoint for listing and creating records:

```
package com.example.demo.controller;

import com.example.demo.entity.Learner;
import com.example.demo.repository.LearnerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/learners")
public class LearnerController {

    @Autowired
    private LearnerRepository learnerRepository;

    @GetMapping
    public List<Learner> listLearners() {
        return learnerRepository.findAll();
    }

    @PostMapping
    public Learner createLearner(@RequestBody Learner learner) {
        return learnerRepository.save(learner);
    }
}
```

11.1 Database Schema for the Quiz Application

Two tables cover the quiz's needs: one holding the question bank, the other logging player results.

```
CREATE TABLE Question (  
  question_id      INT PRIMARY KEY AUTO_INCREMENT,  
  category         VARCHAR(50) NOT NULL,  
  question_text    VARCHAR(255) NOT NULL,  
  option_a         VARCHAR(100),  
  option_b         VARCHAR(100),  
  option_c         VARCHAR(100),  
  option_d         VARCHAR(100),  
  correct_option   CHAR(1) NOT NULL  
);  
  
CREATE TABLE PlayerScore (  
  score_id        INT PRIMARY KEY AUTO_INCREMENT,  
  player_name     VARCHAR(100) NOT NULL,  
  final_score     INT NOT NULL,  
  date_played    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Design rationale:

- The *category* column on Question lets questions about Crops, Geography, and Academic Institutions of the district be queried independently rather than mixed together.
- Four separate option columns (a-d) keep each choice atomic, which is simpler to validate than parsing one combined string of options.
- *correct_option* is stored as a single character so answer-checking is a fast, direct comparison.
- *PlayerScore* is kept as its own table because a player's session result is a distinct entity from a question — this follows normalization and keeps the schema free of redundancy.
- *date_played* defaults to the current timestamp automatically, which supports score history and leaderboard features without any extra application logic.
- Auto-increment primary keys guarantee uniqueness on both tables and keep JDBC insert/retrieve call simple.

11.2 Servlet for Persisting a Score

`SaveScoreServlet.java` — receives the player's final score via POST and writes it to MySQL using a prepared statement:

```
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

public class SaveScoreServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String playerName = request.getParameter("playerName");
        int finalScore = Integer.parseInt(request.getParameter("finalScore"));

        String dbUrl = "jdbc:mysql://localhost:3306/quizdb";
        String dbUser = "root";
        String dbPass = "root";

        String insertSql = "INSERT INTO PlayerScore (player_name, final_score) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(dbUrl, dbUser, dbPass);
            PreparedStatement pstmt = conn.prepareStatement(insertSql)) {

            pstmt.setString(1, playerName);
            pstmt.setInt(2, finalScore);

            int rowsAffected = pstmt.executeUpdate();

            response.setContentType("text/html");
            PrintWriter out = response.getWriter();

            if (rowsAffected > 0) {
                out.println("Score saved successfully for " + playerName);
            } else {
                out.println("Failed to save score.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
            throw new ServletException("Database error", e);
        }
    }
}
```

1.3 Console Quiz Prototype

A small console program that models each multiple-choice question, checks the player's answer, and tallies a final score. This mirrors the categories used in the Tangail-district quiz (Crops, Geography, Institutions).

```
import java.util.*;

class QuizQuestion {
    String category;
    String prompt;
    String[] choices;
    char correctChoice;

    QuizQuestion(String category, String prompt, String[] choices, char correctChoice) {
        this.category = category;
        this.prompt = prompt;
        this.choices = choices;
        this.correctChoice = correctChoice;
    }

    boolean isCorrect(char attempt) {
        return Character.toUpperCase(attempt) == Character.toUpperCase(correctChoice);
    }
}

public class QuizRunner {
    public static void main(String[] args) {
        List<QuizQuestion> bank = new ArrayList<>();

        bank.add(new QuizQuestion("Crop",
            "Which crop is widely cultivated in the district's plains?",
            new String[]{"A) Rice", "B) Coffee", "C) Cactus", "D) Pine"}, 'A'));

        bank.add(new QuizQuestion("Geography",
            "Which river flows through the district?",
            new String[]{"A) Nile", "B) Local River X", "C) Amazon", "D) Danube"}, 'B'));

        bank.add(new QuizQuestion("Institution",
            "Which is a well-known academic institution in the district?",
            new String[]{"A) District University", "B) Foreign College", "C) None", "D) Unknown"}, 'A'));

        Scanner scanner = new Scanner(System.in);
        int score = 0;

        for (QuizQuestion q : bank) {
            System.out.println "[" + q.category + "] " + q.prompt;
            for (String choice : q.choices) {
                System.out.println(choice);
            }
            System.out.print("Enter your answer (A/B/C/D): ");
            char attempt = scanner.next().charAt(0);

            if (q.isCorrect(attempt)) {
                System.out.println("Correct!\n");
                score++;
            } else {
                System.out.println("Wrong! Correct answer: " + q.correctChoice + "\n");
            }
        }

        System.out.println("Final Score: " + score + "/" + bank.size());
        scanner.close();
    }
}
```

12.1 Creational Design Patterns

Pattern	Purpose in one line
Singleton	Guarantees a class has exactly one instance and gives a single, well-known access point to it.
Factory Method	Defines a method for creating an object but lets subclasses decide which concrete class gets instantiated.
Abstract Factory	Offers an interface for producing families of related objects without exposing their concrete classes.
Builder	Splits the construction of a complex object from its final representation, allowing it to be assembled step by step.
Prototype	Produces new objects by cloning an existing prototype rather than building each one from scratch.

12.2 Structural Design Patterns

Pattern	Purpose in one line
Adapter	Wraps a class behind the interface a client expects, letting otherwise incompatible interfaces cooperate.
Bridge	Separates an abstraction from its implementation so each can evolve independently.
Composite	Arranges objects into tree structures that represent part-whole hierarchies, so individual items and groups are treated the same way.
Decorator	Adds new responsibilities to an object at runtime without touching its underlying structure.
Facade	Presents a simplified, single entry point in front of a more complex subsystem.
Flyweight	Shares state across many fine-grained objects to cut memory use when large numbers of them are needed.
Proxy	Stands in for another object and controls access to it.

12.3 Pattern Implementations

Thread-safe Singleton using double-checked locking:

```
public class ConfigManager {  
    private static volatile ConfigManager instance;  
    private ConfigManager() {}  
    public static ConfigManager getInstance() {  
        if (instance == null) {  
            synchronized (ConfigManager.class) {  
                if (instance == null) {  
                    instance = new ConfigManager();  
                }  
            }  
        }  
        return instance;  
    }  
    public void printIdentity() {  
        System.out.println("Singleton instance: " + this.hashCode());  
    }  
    public static void main(String[] args) {  
        ConfigManager first = ConfigManager.getInstance();  
        ConfigManager second = ConfigManager.getInstance();  
  
        first.printIdentity();  
        second.printIdentity();  
  
        System.out.println("Same instance? " + (first == second));  
    }  
}
```

Adapter pattern — converting a 220V supply into a 110V-compatible interface:

```
interface HighVoltageSocket {  
    void supplyHighVoltage();  
}  
  
class WallSocket implements HighVoltageSocket {  
    public void supplyHighVoltage() {  
        System.out.println("Supplying 220V power");  
    }  
}  
  
interface LowVoltageSocket {  
    void supplyLowVoltage();  
}  
  
class VoltageAdapter implements LowVoltageSocket {  
    private HighVoltageSocket source;  
    public VoltageAdapter(HighVoltageSocket source) {  
        this.source = source;  
    }  
    public void supplyLowVoltage() {  
        System.out.println("Adapter converting 220V to 110V");  
        source.supplyHighVoltage();  
    }  
}  
  
public class AdapterDemo {  
    public static void main(String[] args) {  
        HighVoltageSocket wall = new WallSocket();  
        LowVoltageSocket adapter = new VoltageAdapter(wall);  
        adapter.supplyLowVoltage();  
    }  
}
```